

Aula 1 - Modelo booleano

Modelagem:

1. Definir arcabouço lógico para representar documentos e consultas
2. Definir função de correspondência/ranqueamento entre cada documento e uma consulta

Um modelo de RI é uma quádrupla onde:

- D = conjunto de representações dos documentos da coleção
- Q = conjunto de representações de consultas formuladas a partir das necessidades de informação dos usuários
- F = álgebra booleana
- $R(q_i, d_j)$: 1 se doc satisfaz consulta, 0 caso contrário

$D \rightarrow d_j \rightarrow R(q_i, d_j) \quad Q \rightarrow q_i$

Representação de um documento

- Cada documento deve ser representado por um conjunto de **palavras-chave** ou **termos de indexação** (palavra/grupo de palavras consecutivas em um documento)
- Exemplos de termos: palavras, expressões, metadados, etc

Conceitos básicos

- Tokenização (palavra por palavra, bigrama)
- Remoção de stop words (ou, e, de, não)
- Normalização de termos (tudo minúsculo, tratar hífen, remover pontuação, normalização de URLs, etc)
- Lematização (fui, iremos $\rightarrow$ ir)
- Stemming (mudanças, mudamos $\rightarrow$ muda)
- Radicalização (passei, passamos, passou $\rightarrow$ pass)
- Desambiguação (passar recebe um id específico segundo o contexto da frase)

Recuperação baseada em termos de indexação

- Vantagens: Representação simples, indexação eficiente, consultas transparentes
- Desvantagens: Dificuldade em representar contexto/semântica, consultas curtas/ambíguas.

Conceitos básicos

1. $V = \{k_1, \dots, k_t\}$ = Vocabulário após tratamento
2. Sequência de pares: (termo de indexação, docID)
3. Ordenação pelos termos, e depois pelo docID
4. Termos de indexação repetidos em um único documento são combinados
5. Divisão em dicionário e postings (lista de documentos). É uma hash table com índice = termo, e em cada índice uma lista encadeada dos documentos onde aparece
6. Frequência do termo em cada documento é adicionada

Custo de armazenamento

Termos e frequências, ponteiros, listas de docIDs. Quanto mais informação cada posting armazena, maior o custo, mas mais sofisticadas podem ser as consultas

Posting $\rightarrow$ Permite

Somente o docID $\rightarrow$ Consultas booleanas docID + frequência $\rightarrow$ TF-IDF e BM25 docID + posições $\rightarrow$ consultas por frase e proximidade docID + campos $\rightarrow$ busca em título, resumo e corpo

Modelo Booleano

- Uso de NOT, AND, OR na Query.
- Cada documento é visto como um conjunto de palavras

Precedência: () -> NOT -> AND -> OR

Processamento: AND

1. Brutus AND Caesar
2. Localize Brutus no dicionário, recupere sua lista de documentos (postings)
3. Localize Caesar no dicionário, recupere sua lista
4. Combine as duas listas de postings (intersecção)

Outros operadores

- NOT Caesar

$D \Rightarrow \{1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6\}$ Caesar $\Rightarrow \{2 \rightarrow 4\}$

Basta pegar 1,3,5,6 (não têm Caesar)

- Brutus AND NOT Caesar

Ao invés de fazer Brutus AND (NOT Caesar), o que é ineficiente, dá pra fazer isso direto (algoritmo só pro AND NOT)

- Brutus OR NOT Caesar

Aqui não há jeito, tem que lidar com documentos que não existem na lista de documentos de Caesar, ou seja lidar com o universo D.

- A AND B AND C AND D -> Tende a ser mais eficiente executar primeiro as duplas mais raras (conjuntos menores)

Problemas

- Satisfação parcial das condições da consulta (correspondência lógica != relevância). Refinar a consulta ajuda, mas não resolve tudo.
- Não é possível ranqueamento
- Necessidade de traduzir query em expressão booleana

Aula 2 - Modelo probabilístico

No modelo booleano: Documento satisfz a consulta? (sim/não)

No modelo probabilístico: Qual documento tem maior chance de ser relevante para a consulta?

Princípio do ranqueamento probabilístico

Dada uma consulta q , cada documento d_j possui uma probabilidade estimada de ser relevante:

$$P(R = 1|d, q)$$

Então o ranqueamento será:

$$P(R|d_1, q) > P(R|d_2, q) > P(R|d_3, q)$$

Binary Independence Model (BIM)

- Considera apenas presença/ausência de termos
- Assume independência entre os termos dadas as classes relevante/não relevante.

$$\text{sim}(d_j, q) = \frac{P(R|(d_j), q)}{P(R^*|(d_j), q)}$$

- $d_j = (w_{1,j}, w_{2,j}, \dots, w_{t,j})$ - Onde o peso $w_{i,j}$ indica 1 se o termo k_i ocorre no documento d_j e 0 caso contrário.
- R - Conjunto de documentos inicialmente estimados como relevantes
- R^* - Conjunto de documentos não relevantes

Ex: se $P(R|d, q) = 0,8$, então $P(R^*|d, q) = 0,2$ e $\text{odds} = 0, \frac{8}{0}, 2 = 4$. Ou seja, o documento é quatro vezes mais provável de ser relevante do que não relevante.

Ao aplicar a regra de Bayes na equação anterior, temos:

$$\text{sim}(d_j, q) \approx \sum_{k_i \in q \wedge k_i \in d_j} \log\left(\frac{p_i R}{1-p_i R}\right) + \log\left(\frac{1-p_i R^*}{p_i R^*}\right)$$

- $p_i R$ = Probabilidade do termo k_i estar num documento aleatório de R
- $p_i R^*$ = Probabilidade do termo k_i estar num documento aleatório de R^*

Para computar $p_i R$ e $p_i R^*$, definimos:

- N = número de documentos da coleção
- n_i = número de documentos que contém o termo k_i
- R = número de documentos relevantes para a consulta q
- r_i = número de documentos relevantes que contém o termo k_i

$$p_i R = \frac{r_i}{R}$$

$$p_i R^* = \frac{n_i - r_i}{N - R}$$

Assim, podemos reescrever a equação.

$$\text{sim}(d_j, q) = \sum_{k_i \in q \wedge k_i \in d_j} \log\left(\frac{r_i(N - n_i - R + r_i)}{(R - r_i)(n_i - r_i)}\right)$$

E aplica-se uma correção de 0,5 para evitar probabilidades zero ou razões infinitas.

$$\text{sim}(d_j, q) = \sum_{k_i \in q \wedge k_i \in d_j} \log\left(\frac{(r_i + 0.5)(N - n_i - R + r_i + 0.5)}{(R - r_i + 0.5)(n_i - r_i + 0.5)}\right)$$

No caso de ausência quanto à relevância dos documentos, $R = r_i = 0$.

$$\text{sim}(d_j, q) = \sum_{k_i \in q \wedge k_i \in d_j} \log\left(\frac{N - n_i + 0.5}{n_i + 0.5}\right)$$

Nesse caso, temos:

- $n_i < \frac{N}{2}$ -> Peso positivo
- $n_i = \frac{N}{2}$ -> Peso zero
- $n_i > \frac{N}{2}$ -> peso negativo

Termos raros são muito mais discriminativos.

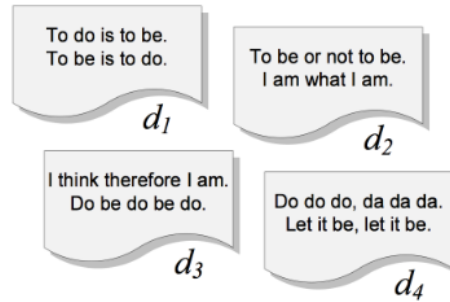
Para evitar o score negativo, o fator $n - i$ do numerador pode ser removido.

$$\text{sim}(d_j, q) = \sum_{k_i \in q \wedge k_i \in d_j} \log\left(\frac{N + 0.5}{n_i + 0.5}\right)$$

- N = número de documentos da coleção
- n_i = número de documentos que contém o termo k_i

Exemplo

Coleção:



Consulta: "to do"

- "to" aparece em 2 documentos.
- "do" aparece em 3 documentos.

$$\text{sim}(d_1, q) = \log\left(\frac{4 + 0.5}{2 + 0.5}\right) + \log\left(\frac{4 + 0.5}{3 + 0.5}\right) = 0.364$$

- $\text{sim}(d_2, q) = 0.255$
- $\text{sim}(d_3, q) = 0.109$
- $\text{sim}(d_4, q) = 0.109$

$$d_1 > d_2 > d_3 == d_4$$

Supondo que, uma vez retornados os documentos ao usuário na ordem acima, ele tenha selecionado d_1, d_3 como relevantes. Recalcule as similaridades a partir desse feedback.

- $R = 2$
- $n_{\text{to}} = 2$
- $r_{\text{to}} = 1$
- $n_{\text{do}} = 3$
- $r_{\text{do}} = 2$

$$w_i = \log\left(\frac{(r_i + 0.5)(N - n_i - R + r_i + 0.5)}{(R - r_i + 0.5)(n_i - r_i + 0.5)}\right)$$

Peso de "to": 0 Peso de "do": 0.69897

Score de cada documento:

- $\text{score}(d_1) = w_{\text{to}} + w_{\text{do}} = 0.69897$
- $\text{score}(d_2) = w_{\text{to}} = 0$
- $\text{score}(d_3) = w_{\text{do}} = 0.69897$
- $\text{score}(d_4) = w_{\text{do}} = 0.69897$

Vantagens e desvantagens

Vantagens

- Documentos são ranqueados de acordo com sua probabilidade de relevância

Desvantagens

- Falta de normalização dos documentos
- Não utilização da frequência dos termos
- Necessidade de estimar uma separação inicial dos documentos

BM25

A função de ranqueamento do BM25 é:

$$\text{score}(d, q) = \sum_{t \in q} \text{IDF}(t) \frac{tf(t, d)(k_1 + 1)}{(tf(t, d) + k_1) \left(1 - b + \frac{b(|d|)}{\text{avgdl}}\right)}$$

O modelo considera 3 aspectos fundamentais:

- Raridade -> Termos muito frequentes carregam pouca informação
- Frequência
- Tamanho

Aula 3 - Modelo Vetorial

Ponderação TF-IDF

Considera:

- Frequência do termo (TF): importância do termo dentro do documento
- Frequência inversa de documento (IDF): capacidade do termo de discriminar documentos na coleção
- Considera importância local + importância global

Ponderação de frequência dos termos

- Hipótese de Luhn - O peso $tf_{i,j}$ de um termo k_i que ocorre em um documento d_j é proporcional à frequência do termo $f_{i,j}$
- Assim ; $tf_{i,j} = f_{i,j}$
- Variante:

$$tf_{i,j} = \begin{cases} 1 + \log(f_{i,j}) & \text{se } f_{i,j} > 0 \\ 0 & \text{caso contrário} \end{cases}$$

A base do log muda a escala, mas não a ordem relativa dos pesos.

Ponderação pela frequência inversa dos documentos

- n_i = Número de documentos em que o termo k_i ocorre na coleção
- N = Número total de documentos

$$idf_i = \log\left(\frac{N}{n_i}\right)$$

Ponderação TF-IDF

- $w_{i,j}$ = Ponderação associada com o termo k_i e documento d_j

TF-IDF é definido como:

$$w_{i,j} = \begin{cases} (1 + \log f_{i,j}) \cdot \log\left(\frac{N}{n_i}\right) & \text{se } f(i,j) > 0 \\ 0 & \text{Caso contrário} \end{cases}$$

Normalização pelo tamanho dos documentos

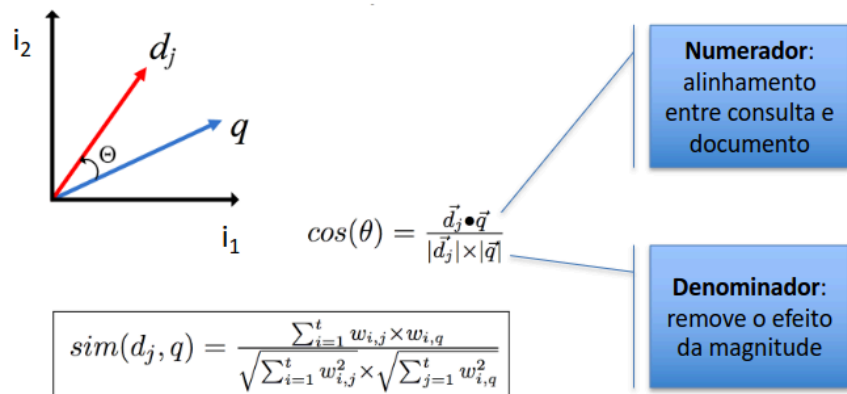
- Tamanhos de documentos variam consideravelmente
- Problema: Documentos longos tendem a acumular maiores pesos e têm maior chance de compartilhar termos com a consulta

Fazemos a divisão do vetor de termos ponderados pela norma.

- Documentos: $\vec{d}_j = w_{i,j} \cdot (k_i)$
- Norma do documento d_j : $|\vec{d}_j| = \sqrt{\sum_i^t w_{i,j}^2}$
- Normalização: $\hat{d}_j = \frac{d_j}{|\vec{d}_j|}$

Ranqueamento

- Pesos do documento: $\vec{d}_j = (w_{1,j}, w_{2,j}, \dots, w_{t,j})$
 - Pesos da query: $\vec{q} = (w_{1,q}, w_{2,q}, \dots, w_{t,q})$
 - Pesos TF-IDF: $w_{i,q} = (1 + \log f_{i,q}) \cdot \log\left(\frac{N}{n_i}\right)$
 - Pesos TF-IDF: $w_{i,j} = (1 + \log f_{i,j}) \cdot \log\left(\frac{N}{n_i}\right)$
- Similaridade entre um documento d_j e uma consulta q



- Uma vez que $w_{ij} > 0$ e $w_{iq} > 0$, temos que $0 \leq \text{sim}(d_j, q) \leq 1$

Vantagens e desvantagens

Vantagens:

- Ponderação de termos melhora a qualidade do conjunto-resposta
- Casamento parcial permite a recuperação de documentos que se aproximam da consulta
- Fórmula de cosseno ordena documentos de acordo com um grau de similaridade em relação à consulta
- Normalização pelo tamanho do documento já é realizada pela própria função de cosseno

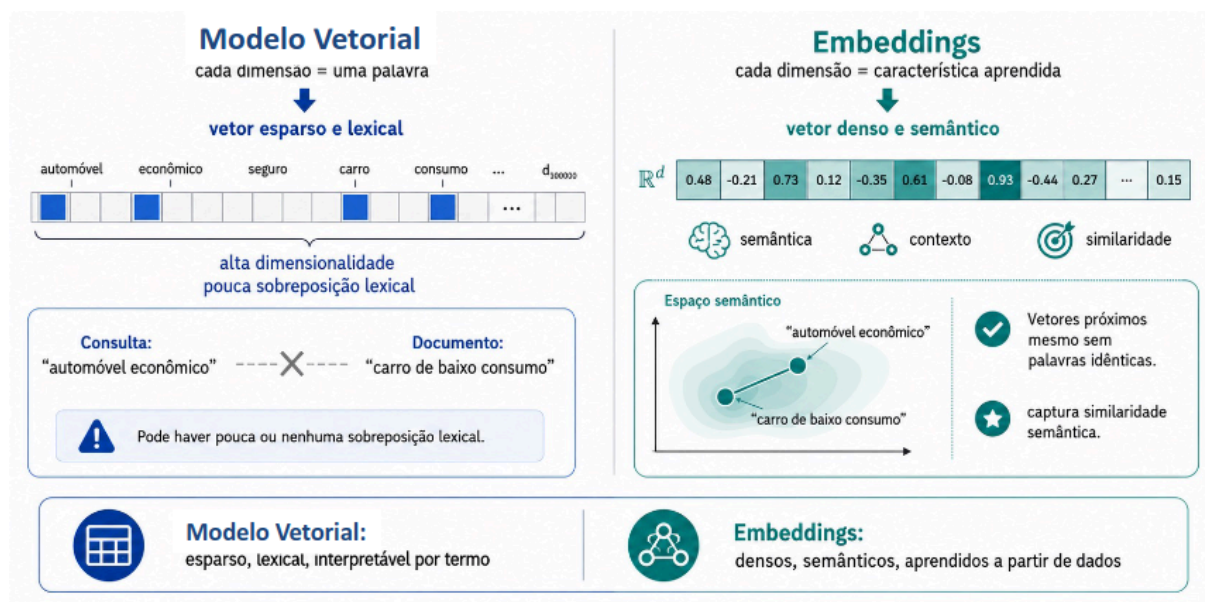
Desvantagens:

- Assume-se independência dos termos de indexação
- Polissemia e sinonímia
- Ignora ordem/posição das palavras
- Vocabulário muito grande e esparso

Principal limitação

- Consulta: Automóvel econômico
- Documento: Carro de baixo consumo (não retornado)

Resolvemos isso melhorando a representação de documentos por **embeddings**.



Avaliação da Recuperação

- Mensurar o quão bem o sistema atende à necessidade de informação do usuário

Níveis de avaliação

- Avaliação offline - precision, recall, MAP, MRR, nDCG, etc
- Estudos com usuários - satisfação, esforço, tempo, utilidade, etc
- Avaliação online - Cliques, CTR, A/B testing, interleaving, etc

Precision & Recall

Considere R = conjunto de documentos relevantes, A = Conjunto resposta.

- Recall: Fração de documentos relevantes que foram recuperados

$$\text{Recall} = \frac{|R \cap A|}{|R|}$$

- Precision: Fração de documentos recuperados que são relevantes

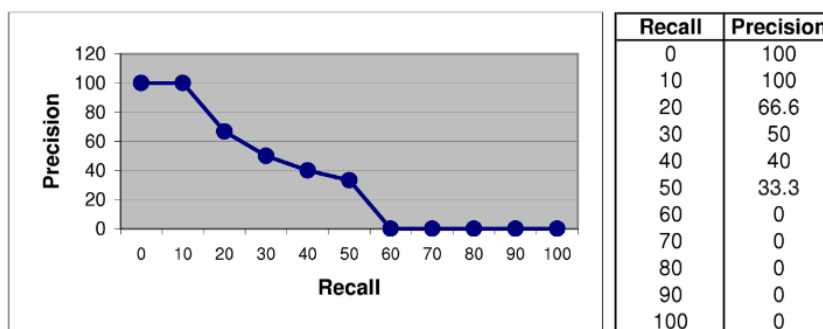
$$\text{Recall} = \frac{|R \cap A|}{|A|}$$

As métricas de precision e recall assumem que todos os documentos no conjunto A foram examinados. Porém, o sistema não retorna todos os documentos do conjunto A de uma vez. É fornecido um subset ranqueado, onde os documentos são examinados a partir do topo.

Considere uma busca onde A possui 10 documentos relevantes, mas os 15 primeiros apresentam documentos aleatórios.

- Documento d_{123} , primeiro da lista, é relevante
 - Corresponde a 10% de todos os documentos relevantes
 - Assim, neste momento temos precisão = 100% e revocação = 10%
- Documento d_{56} , terceiro da lista, é o próximo relevante
 - 2/3 dos documentos são relevantes, e 2/10 dos documentos relevantes foram retornados
 - Assim, temos precisão = 66,6% e revocação = 20%

01. d_{123} • 06. d_9 • 11. d_{38}
 02. d_{84} 07. d_{511} 12. d_{48}
 03. d_{56} • 08. d_{129} 13. d_{250}
 04. d_6 09. d_{187} 14. d_{113}
 05. d_8 10. d_{25} • 15. d_3 •



Para plotar os resultados usando os valores-padrão de recall (0, 10%, 20%, etc...) é necessário interpolação. Assim, consideramos $P(0.1) = P(0.09)$ caso $P(1)$ não exista.

Precision in K (P@k)

Em busca Web, altos valores de recall não são importantes. O importante é a alta quantidade de documentos relevantes no topo do ranking.

$P@k$ mede o desempenho quando K documentos foram examinados.

$$P@k = \frac{\text{Relevantes}}{\text{Não Relevantes}}, \text{ até } k$$

Hit Rate / Success (Hit@k)

Útil quando basta que apenas um resultado correto seja recuperado.

$$\text{Hit}@k = \begin{cases} 1, & \text{se existe um relevante no top-}k \\ 0, & \text{caso contrário} \end{cases}$$

MAP (MAP@k)

Média dos valores de precisão obtidos após cada novo documento relevante observado.

$$\text{MAP} = \frac{1}{|Q|} \sum_q \text{AP}(q)$$

- q = Query
- Q = Total de queries

$$AP@k(q) = \frac{1}{\min(|R_q|, K)} \sum_{k=1}^K P@k \cdot \text{rel}(k)$$

- R_q = Número de documentos relevantes na query.
- $\text{rel}(k)$ = 1 caso relevante, 0 caso contrário.

MRR

Quando interessa principalmente o primeiro resultado relevante. Somente ele é analisado.

$$MRR = \frac{1}{|Q|} \sum_q RR(q)$$

- q = Query
- Q = Total de queries

$$RR(q) = \frac{1}{\text{rank}_q}$$

- rank_q = Posição do primeiro relevante. (posição 4 - $RR = 0.25$)

DCG

Ganho acumulado descontado. DCG considera diferentes níveis de relevância de documentos durante a avaliação. Mede se documentos mais relevantes aparecem cedo no ranking.

Cada posição recebe um ganho de acordo com a relevância (rel_i). Por ex:

Ranking = [3, 0, 2, 1]

Ganho = [7, 0, 3, 1]

$$DCG@k = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}$$

- i = Posição do documento (a partir de 1)
- rel_i = Ganho (+ ou - relevante)

iDCG

DCG com melhor ranking possível (ordena do mais relevante pro menos relevante e aplica DCG).

nDCG@k

É a métrica final.

$$nDCG@k = \frac{DCG@k}{IDCG@k}$$